



Caio Gabriel Pereira de Menezes Correia
Caio Renato dos Santos Claudino
José Francisco de Araújo Neto

COMPILADORES – PROJETO FINAL

VALIDADOR DE BOOTSTRAPING

MACEIÓ - ALAGOAS

2026

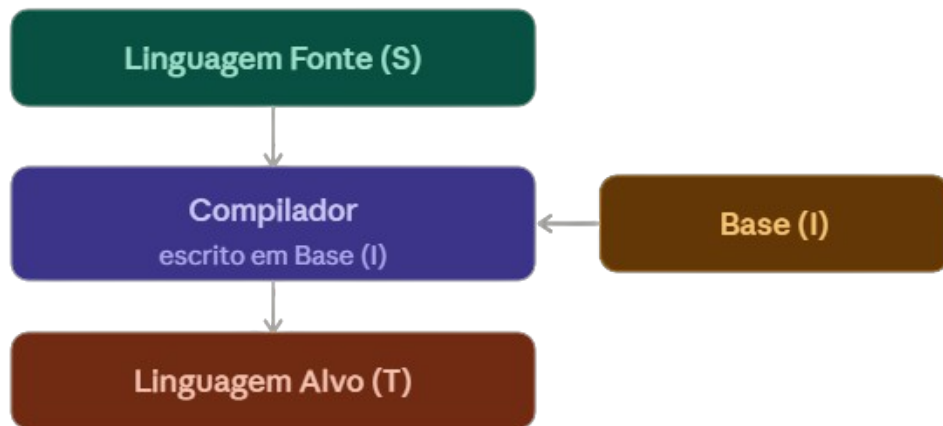
1. Introdução

1.1. Diagramas em T

Diagramas em T são notações utilizadas na teoria de compiladores para representar processos de tradução de linguagens. Consistem em três componentes:

- **Linguagem Fonte (S)**: Linguagem a ser compilada
- **Linguagem Alvo (T)**: Linguagem resultante da compilação
- **Linguagem Base (I)**: Linguagem em que o compilador está escrito

Representação visual:



A regra fundamental: para que um compilador seja executável, a linguagem Base deve estar disponível no ambiente.

1.2. Problema que o validador resolve

O **bootstrap de compiladores** é um problema clássico: como implementar um compilador para uma linguagem quando esse compilador precisa ser escrito naquela mesma linguagem? O Validador de Bootstrapping automatiza a verificação de viabilidade deste processo, identificando se um diagrama é executável e onde um encadeamento falha.



O validador identifica exatamente em qual etapa o encadeamento falha e qual linguagem base estava ausente no ambiente, fornecendo uma justificativa clara para cada resultado.

2. Metodologia de implementação

2.1. Linguagem de ambiente

O projeto foi desenvolvido em **Python 3.10**, utilizando exclusivamente bibliotecas da biblioteca padrão da linguagem. Essa escolha elimina dependências externas e garante portabilidade entre sistemas operacionais sem necessidade de instalação adicional.

2.2. Estrutura do projeto

O código foi organizado em três arquivos com responsabilidades bem definidas:

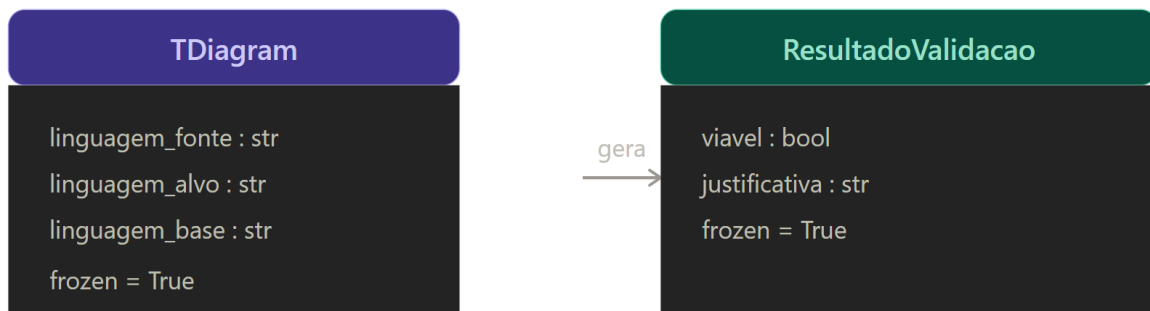


- **models.py** — contém todas as estruturas de dados e a lógica de validação. É o núcleo do projeto, sem dependência dos demais arquivos;
- **main.py** — responsável pela interface com o usuário via terminal, lendo os dados de entrada e exibindo os resultados;
- **validador.py** — executa casos de teste pré-definidos para demonstrar e verificar o comportamento do sistema.

2.3. Modelagem das Estruturas de Dados

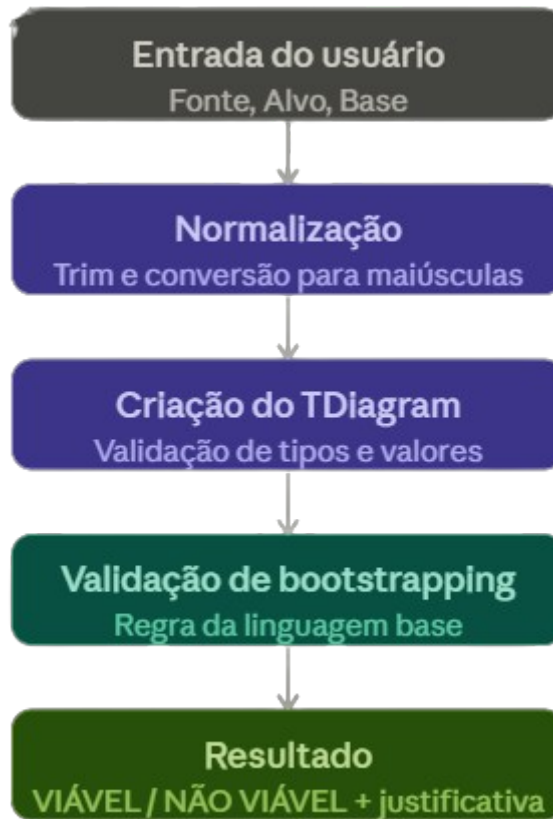
A representação dos diagramas em T foi implementada utilizando **dataclasses** com o atributo **frozen=True**, o que garante **imutabilidade** após a criação do objeto. Isso evitará alterações acidentais nos dados durante o processo de validação.

Duas estruturas principais foram definidas:



2.4. Fluxo de Validação

O **processo de validação** segue um fluxo bem definido, desde a entrada dos dados até a exibição do resultado:



2.5. Regras de Validação Implementadas

validar_bootstraping — avalia um único diagrama em T. Verifica se a linguagem base coincide com a linguagem disponível no ambiente. Caso coincidam, o processo é considerado viável e uma justificativa é gerada. Caso contrário, retorna inviável informando a incompatibilidade.

validar_encadeamento — recebe uma sequência de diagramas e uma linguagem inicial. A cada etapa, a linguagem alvo do diagrama atual torna-se a linguagem disponível para o próximo. Se qualquer etapa falhar, o processo é interrompido e a etapa com falha é identificada na justificativa.

2.6. Tratamento de entradas inválidas

O sistema aplica validações em múltiplas camadas para garantir robustez:

- Verificação de tipo: a entrada deve ser uma string;
- Verificação de conteúdo: a string não pode ser vazia ou conter apenas espaços;
- Normalização: remoção de espaços e conversão para maiúsculas antes de qualquer comparação;
- Propagação de erros: exceções do tipo ***ValueError*** e ***TypeError*** são capturadas na camada de interface e exibidas com mensagens claras ao usuário.

3. Casos de Testes

Os três casos a seguir são executados automaticamente ao rodar ***python validador.py***. Os prints a seguir mostrarão as saídas ao rodar o arquivo em conjunto com as saídas geradas pelo arquivo ***python main.py***

3.1. Caso viável (validador.py/main.py)

```
=====
Caso viavel
Linguagem inicial disponivel: C
Entrada:
  T1: fonte=C, alvo=ASM, base=C
  T2: fonte=MINILANG, alvo=C, base=ASM
Resultado: VIAVEL
Justificativa: Todas as etapas respeitam a regra dos T-diagrams: a linguagem base de cada compilador coincide com a linguagem disponivel naquele ponto do encadeamento.
=====
```

```
=====
Linguagem Fonte: C
Linguagem Alvo: ASM
Linguagem Base: C

Diagrama informado:
  Fonte: C
  Alvo: ASM
  Base: C

Resultado: VIAVEL
Justificativa: Existe um compilador escrito/executavel em C capaz de traduzir C para ASM. Como a base coincide com a linguagem disponivel, a compilacao cruzada e viavel.
=====
```

A linguagem inicial disponível é C. O primeiro diagrama (T1) usa C como base — compatível. O alvo de T1 é ASM, que se torna a linguagem disponível para T2. O segundo diagrama (T2) usa ASM como base — compatível. Encadeamento aprovado.

3.2. Caso inviável

```
=====
Caso inviavel
Linguagem inicial disponivel: Python
Entrada:
  T1: fonte=PYTHON, alvo=C, base=PYTHON
  T2: fonte=MINILANG, alvo=ASM, base=RUST
Resultado: NAO VIAVEL
Justificativa: Etapa 2 falhou. Existe um compilador escrito/executavel em PYTHON capaz de traduzir PYTHON para C. Como a base coincide com a linguagem disponivel, a compilacao cruzada e via
vel. A linguagem base do compilador precisa existir no ambiente disponivel. Esperado: C. Recebido: RUST.
=====
```

```
Validador de Bootstrapping com Diagramas em T
-----
Linguagem Fonte: Python
Linguagem Alvo: C
Linguagem Base: Rust

Diagrama informado:
  Fonte: PYTHON
  Alvo: C
  Base: RUST

Resultado: NAO VIAVEL
Justificativa: A linguagem base do compilador precisa existir no ambiente disponivel. Esperado: PYTHON. Recebido: RUST.
```

T1 é válido — a base Python coincide com a linguagem disponível. Após T1, a linguagem disponível passa a ser C. T2 exige Rust como base, mas apenas C está disponível. O sistema identifica a falha na Etapa 2 e interrompe a validação.

3.3. Caso de entrada inválida

```
Validador de Bootstrapping com Diagramas em T
-----
Linguagem Fonte:

Entrada invalida: Linguagem Fonte nao pode ficar vazia.
```

Esse caso demonstra que o sistema possui validação de entrada e não permite que o processo avance com dados incompletos.